

Structures de données usuelles

Partie II

1. Les ensembles

1.1 Définition

Definition 1.1.1 (Ensemble - Set)

Un ensemble (*set* en anglais), est une collection stockant des éléments, sans ordre particulier et sans répétition.

1.1 Définition

Definition 1.1.1 (Ensemble - Set)

Un ensemble (*set* en anglais), est une collection stockant des éléments, sans ordre particulier et sans répétition.

L'implémentation classique de cette structure de données utilise ce qu'on appelle une **fonction de hachage**.

1.1 Définition

Definition 1.1.1 (Ensemble - Set)

Un ensemble (*set* en anglais), est une collection stockant des éléments, sans ordre particulier et sans répétition.

L'implémentation classique de cette structure de données utilise ce qu'on appelle une **fonction de hachage**.



Important

Les éléments doivent être *hashable*, c'est à dire qu'ils doivent être **immuable**.

Par exemple les listes ne sont pas *hashable* ...

1.2 Opérations

Voici la liste des opérations phares qu'implémente cette structure de données :

Opération	Complexité moyenne
Ajout d'un élément	$\Theta(1)$
Retrait d'un élément	$\Theta(1)$
Présence d'un élément	$\Theta(1)$

1.2 Opérations

Voici la liste des opérations phares qu'implémente cette structure de données :

Opération	Complexité moyenne
Ajout d'un élément	$\Theta(1)$
Retrait d'un élément	$\Theta(1)$
Présence d'un élément	$\Theta(1)$

Voici les implémentations disponibles des *hashset* dans certains langages de programmation :

- `set` en Python.
- `HashSet` en Java.
- `std::unordered_set` en C++.

Les opérations ont une complexité moyenne, vous vous doutez bien que quelque chose peut mal tourner ...

1.3 Cas critique

Le problème de l'implémentation de cette structure, c'est qu'on peut avoir des **problèmes de collisions**, donc nos opérations seront en $\mathcal{O}(n)$.

Note

On parle de **problème de collisions**, lorsque notre fonction de hachage attribue le même emplacement pour plusieurs éléments.

1.3 Cas critique

Le problème de l'implémentation de cette structure, c'est qu'on peut avoir des **problèmes de collisions**, donc nos opérations seront en $\mathcal{O}(n)$.

Note

On parle de **problème de collisions**, lorsque notre fonction de hachage attribue le même emplacement pour plusieurs éléments.

Pour régler ce problème, on peut *chiffrer* nos valeurs pour les disperser, par exemple dans le cas de nombre :

```
from random import getrandbits

X = getrandbits(32) # Génération d'une clef aléatoire de 32 bits

v = set()
v.add(12 ^ X) # On chiffre avec l'opérateur xor '^'
x = v.pop() # On récupère l'élément chiffré
r = x ^ X # On déchiffre la valeur
```

2. Table de hachage et multi-ensemble

2.1 Définition

Definition 2.1.1 (Table de hachage)

Une **table de hachage** (*hashmap* en anglais) est une collection non ordonnée, où chaque élément (la *clef*) est associé à une valeur. Comme les **ensembles**, les clefs sont distinctes.

On retrouve cette structure dans différent langages de programmation :

- dict en Python (dictionnaire)
- HashMap en Java
- `std::unordered_map` ou `std::map` en C++

2.1 Définition

Definition 2.1.1 (Table de hachage)

Une **table de hachage** (*hashmap* en anglais) est une collection non ordonnée, où chaque élément (la *clef*) est associé à une valeur. Comme les **ensembles**, les clefs sont distinctes.

On retrouve cette structure dans différent langages de programmation :

- dict en Python (dictionnaire)
- HashMap en Java
- `std::unordered_map` ou `std::map` en C++

Definition 2.1.2 (Multi-ensemble)

Un **multi-ensemble** (*multiset* en anglais), est une collection stockant des éléments, sans ordre particulier et **avec répétition**.

Cette structure existe en C++ (`std::multiset`), ceci dit ce n'est pas le cas en Python et Java, nous allons voir comment la récréer ...

2.2 Lien entre multi-ensemble et table de hachage

On peut définir un **multi-ensemble** grâce à une **table de hachage**. Les clefs seront les valeurs distinctes de celui-ci, et les valeurs seront le nombre d'occurrences.

2.2 Lien entre multi-ensemble et table de hachage

On peut définir un **multi-ensemble** grâce à une **table de hachage**. Les clefs seront les valeurs distinctes de celui-ci, et les valeurs seront le nombre d'occurrences.

```
from collections import defaultdict

# `defaultdict` c'est un dictionnaire qui met une valeur par défaut
# lorsqu'une clef n'existe pas. (par exemple pour `int` c'est 0)
multiset = defaultdict(int)
l = [0, 1, 1, 1, 6, 8, 12, 6, 8, 8, 8]
for value in l:
    multiset[value] += 1
print(multiset) # affiche {0:1, 1:3, 6:2, 8:4, 12:1}
```

Les opérations proposées pour ces structures sont les mêmes que les ensembles ...

3. Crédits

Auteur et images : Bilel MAHDJOUR
Pour le **Bureau des technologies** de
Polytech Nice Sophia

4. Sources

- [USACO guide - Introduction to sets & maps](#)
- [CP algorithms](#)